ESP32 mit MQTT

Verbindungsaufbau, Bibliotheken und Ablauf im SmartHome-Projekt

PubSubClient • WiFiClientSecure • HiveMQ Cloud • März 2026

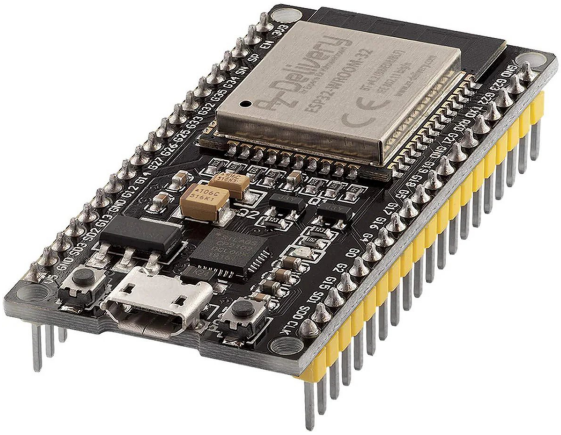

Abb. 1: ESP32-WROOM-32 Entwicklungsboard (AZ-Delivery) — verwendet im SmartHome-Projekt

---

1. Der ESP32 im Projekt

Der ESP32 ist ein leistungsfähiger Mikrocontroller mit integriertem WLAN und Bluetooth. Im SmartHome-Projekt übernimmt er die Rolle eines programmierbaren Aktors und Sensors — er empfängt Schaltbefehle aus der Lazarus-Anwendung und kann gleichzeitig Messwerte (Temperatur, Luftfeuchte) an HiveMQ Cloud senden.

Programmiert wird der ESP32 über die Arduino IDE mit C++. Die Verbindung zu HiveMQ Cloud erfolgt verschlüsselt über TLS auf Port 8883 — identisch wie beim Shelly 1PM Mini Gen3.

Eigenschaft	Wert
`Mikrocontroller`	Espressif ESP32-WROOM-32 (Dual-Core, 240 MHz)
`WLAN`	802.11 b/g/n (2.4 GHz)
`Programmierumgebung`	Arduino IDE mit ESP32-Boardpaket
`GPIO-Pins`	34 programmierbare Pins — LED an GPIO 32
`MQTT-Broker`	HiveMQ Cloud, Port 8883 (TLS)
`Betriebsspannung`	3,3 V (intern), 5 V über USB oder VIN

---

2. Verwendete Bibliotheken

Für die MQTT-Kommunikation über TLS werden zwei Bibliotheken benötigt, die beide über den Arduino Library Manager oder das ESP32-Boardpaket verfügbar sind.

2.1 WiFiClientSecure

Diese Bibliothek ist Teil des offiziellen ESP32-Boardpakets und muss nicht separat installiert werden. Sie stellt eine verschlüsselte TCP-Verbindung bereit — die Grundlage für den TLS-Handshake mit HiveMQ Cloud.

Eigenschaft	Beschreibung
`Bibliothek`	WiFiClientSecure
`Quelle`	Im ESP32-Boardpaket enthalten — kein separater Download
`Einbindung`	#include <WiFiClientSecure.h>
`Aufgabe`	Stellt TLS-gesicherte TCP-Verbindung zu HiveMQ bereit
`Zertifikat`	setInsecure() — kein CA-Zertifikat, nur Verschlüsselung

setInsecure() — Parallele zum Shelly:
Genau wie beim Shelly 1PM Mini Gen3 (TLS no validation) wird auch beim ESP32 die Zertifikatsprüfung mit espClient.setInsecure() deaktiviert.
Die Verbindung ist trotzdem vollständig verschlüsselt.
Beide Geräte lösen damit dasselbe SNI-Problem mit HiveMQ Cloud.

2.2 PubSubClient

Die PubSubClient-Bibliothek von Nick O'Leary ist die am weitesten verbreitete MQTT-Bibliothek für Arduino und ESP32. Sie implementiert das MQTT 3.1.1 Protokoll und unterstützt Publish, Subscribe und automatische Keep-Alive-Pakete.

Eigenschaft	Beschreibung
`Bibliothek`	PubSubClient
`Autor`	Nick O'Leary (knolleary)
`Installation`	Arduino IDE → Bibliotheken verwalten → 'PubSubClient' suchen
`Einbindung`	#include <PubSubClient.h>
`MQTT-Version`	MQTT 3.1.1
`Keep-Alive`	Automatisch — hält die HiveMQ-Verbindung aktiv
`QoS`	QoS 0 (Fire and Forget) — ausreichend für SmartHome

Wichtig — Bibliothek installieren:
PubSubClient muss manuell installiert werden:
Arduino IDE → Werkzeuge → Bibliotheken verwalten →
Suche: 'PubSubClient' → Version von Nick O'Leary installieren

WiFiClientSecure ist bereits im ESP32-Boardpaket enthalten.

3. Verbindungsaufbau — Schritt für Schritt

Der Verbindungsaufbau vom ESP32 zu HiveMQ Cloud läuft in drei klar getrennten Phasen ab. Jede Phase baut auf der vorherigen auf.

1	**WLAN verbinden**	Der ESP32 verbindet sich mit dem lokalen WLAN-Router über SSID und Passwort. Solange keine Verbindung besteht, wartet er in einer Schleife.
2	**TLS konfigurieren**	espClient.setInsecure() deaktiviert die Zertifikatsprüfung. Die Verbindung bleibt verschlüsselt — HiveMQ Cloud wird ohne SNI-Fehler akzeptiert.
3	**MQTT-Server festlegen**	client.setServer() hinterlegt Hostname und Port 8883. Der Callback für eingehende Nachrichten wird mit client.setCallback() registriert.
4	**MQTT verbinden**	client.connect() sendet das MQTT CONNECT-Paket mit Client-ID, Benutzername und Passwort an HiveMQ. Bei Erfolg: CONNACK mit Return Code 0.
5	**Topics abonnieren**	Nach erfolgreichem Connect wird client.subscribe() für das Steuerungs-Topic aufgerufen. Ab sofort leitet HiveMQ Nachrichten an den ESP32 weiter.
6	**Dauerbetrieb (Loop)**	client.loop() muss regelmäßig aufgerufen werden — er verarbeitet eingehende Nachrichten und sendet Keep-Alive-Pakete an HiveMQ.
7	**Reconnect bei Verbindungsverlust**	Wenn client.connected() False zurückgibt, wird automatisch reconnect() aufgerufen. Die Funktion wiederholt den Verbindungsversuch bis HiveMQ antwortet.

---

4. MQTT Topics im Projekt

Der ESP32 kommuniziert mit der Lazarus-Anwendung über fest definierte Topics. Lazarus sendet Befehle, der ESP32 sendet Status-Rückmeldungen — so bleibt die Anzeige in der Lazarus-UI immer aktuell.

	Topic	Payload	Bedeutung
→	`esp32/befehl`	`"ON" / "OFF"`	Schaltbefehl von Lazarus an ESP32
←	`esp32/sensor/led`	`"ON" / "OFF"`	ESP32 meldet aktuellen LED-Status zurück
←	`esp32/sensor/status`	`"online"`	ESP32 meldet sich alle 10 Sekunden als aktiv
←	`esp32/sensor/temperatur`	`"21.5"`	Temperaturwert in °C (für zukünftige Erweiterung)
←	`esp32/sensor/luftfeuchte`	`"55"`	Luftfeuchte in % (für zukünftige Erweiterung)

4.1 Wichtiges Prinzip — Rückmeldung

Der ESP32 sendet nach jedem Schaltvorgang den neuen Status zurück an HiveMQ. Die Lazarus-Anwendung empfängt diese Rückmeldung und aktualisiert die Statusanzeige im Panel. Damit ist sichergestellt, dass die UI immer den echten Zustand des GPIO-Ausgangs zeigt.

Ablauf beim Schalten:

1. Lazarus sendet: esp32/befehl = 'ON'
2. HiveMQ leitet weiter an ESP32
3. ESP32 empfängt → setzt GPIO 32 auf HIGH → LED leuchtet
4. ESP32 sendet: esp32/sensor/led = 'ON'
5. HiveMQ leitet weiter an Lazarus
6. Lazarus empfängt → Status-Panel wechselt auf Grün / 'EIN'

---

5. Dauerbetrieb & Stabilität

5.1 Keep-Alive

MQTT-Verbindungen haben einen Timeout. Wird innerhalb des vereinbarten Keep-Alive-Intervalls keine Nachricht gesendet, trennt der Broker die Verbindung. PubSubClient sendet automatisch PINGREQ-Pakete um die Verbindung aufrechtzuerhalten — solange client.loop() regelmäßig aufgerufen wird.

Parameter	Wert / Verhalten
`Keep-Alive Intervall`	60 Sekunden (Standard PubSubClient)
`client.loop()`	Muss in jedem Durchlauf der loop()-Funktion aufgerufen werden
`Status-Publish`	Alle 10 Sekunden sendet ESP32 esp32/sensor/status = 'online'
`Reconnect`	Automatisch bei Verbindungsverlust (in reconnect()-Funktion)
`Reconnect-Wartezeit`	5 Sekunden zwischen Verbindungsversuchen

5.2 Verhalten bei Verbindungsverlust

Fällt die WLAN- oder HiveMQ-Verbindung aus, erkennt der ESP32 dies über client.connected(). Er versucht dann automatisch die Verbindung wiederherzustellen. Der letzte GPIO-Zustand bleibt dabei unverändert — die LED behält ihren Zustand bis ein neuer Befehl ankommt.

Hinweis:
Während des Reconnect-Versuchs reagiert der ESP32 auf keine eingehenden Befehle. Nach erfolgreicher Wiederverbindung wird das Steuerungs-Topic erneut abonniert.

---

6. GPIO-Ausgang — LED an Pin 32

Im Projekt ist eine LED an GPIO Pin 32 des ESP32 angeschlossen. Dieser Pin dient als Aktor — er wird durch Befehle aus der Lazarus-Anwendung über HiveMQ gesteuert.

Eigenschaft	Beschreibung

`GPIO-Pin`	32
`Funktion`	Digitaler Ausgang (OUTPUT)
`HIGH (3,3 V)`	LED leuchtet — entspricht Befehl 'ON'
`LOW (0 V)`	LED aus — entspricht Befehl 'OFF'
`Startzustand`	LOW (AUS) beim Einschalten des ESP32
`Erweiterbar`	Statt LED kann auch ein Relais-Modul angeschlossen werden

Erweiterungsmöglichkeit:
GPIO 32 kann statt einer LED auch ein 5V-Relaismodul ansteuern.
Das Relaismodul schaltet dann 230V-Verbraucher — z.B. eine Lampe.
Die Lazarus-Anwendung und der MQTT-Code bleiben dabei unverändert.

---

7. Vergleich — Shelly 1PM Mini vs ESP32

Beide Geräte sind über HiveMQ Cloud mit der Lazarus-Anwendung verbunden. Sie ergänzen sich im Projekt gegenseitig:

Merkmal	Shelly 1PM Mini Gen3	ESP32
Programmierung	Fertiggerät — keine Programmierung	Frei programmierbar (Arduino C++)
Montage	In Unterputzdose / Steckdose	Auf Steckplatine / eigene Schaltung
Leistungsmessung	Ja — Watt, Volt, Ampere	Nein (nur mit externem Sensor)
GPIO-Ausgänge	1 Relais (230V-fähig)	34 Pins (3,3V, max. 40mA)
TLS-Verbindung	TLS no validation (Firmware)	setInsecure() (Arduino Code)
MQTT-Topics	Gen3 JSON / RPC-Protokoll	Einfache Strings (ON/OFF)
Einsatzgebiet	230V Verbraucher schalten	Sensoren, LEDs, eigene Hardware
Kosten	ca. 15–20 €	ca. 5–10 €

---

8. Zusammenfassung

Der ESP32 im SmartHome-Projekt:

Bibliothek WiFiClientSecure — TLS-Verbindung (im ESP32-Paket enthalten)
Bibliothek PubSubClient — MQTT 3.1.1 (separat installieren)
espClient.setInsecure() — wie Shelly: TLS ohne Zertifikatsprüfung
HiveMQ Cloud Port 8883 — identische Verbindung wie Shelly
Automatischer Reconnect — stabil auch bei kurzen WLAN-Aussetzern
Rückmeldung über MQTT — Lazarus-UI zeigt immer echten Zustand

Erweiterbar um: BME280-Sensor (Temp/Feuchte), Relais-Modul,
weitere GPIO-Pins, Analogeingänge und vieles mehr.